

MyHealthReport

Platform Structure & Execution Plan

Website, desktop and mobile architecture, technology stack, feature scope, and the medical department's execution plan

28 August 2026

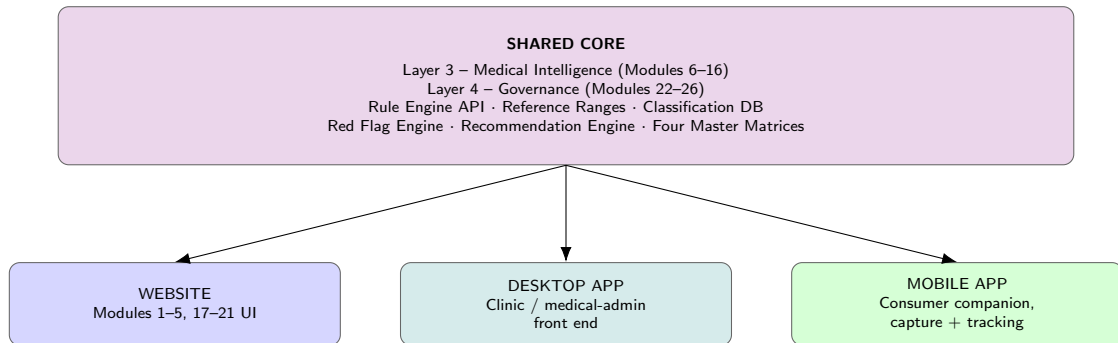
Agenda

- 1 One Core, Three Front Ends
- 2 Website Version
- 3 Desktop App Version
- 4 Mobile App Version
- 5 Suggested Build Sequence
- 6 Medical Department Execution Plan

One Core, Three Front Ends

A single source of truth behind every channel

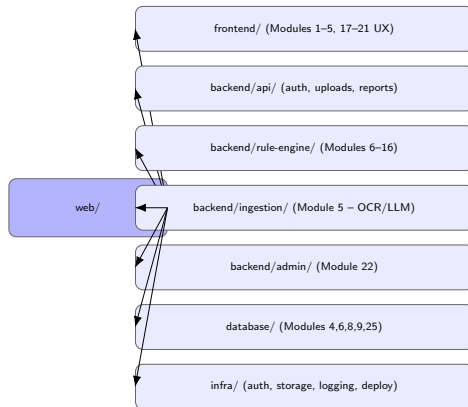
- Medical knowledge is never hard-coded into a client – all three versions are front ends over the *same* backend
- Website, desktop, and mobile call the same Rule Engine API and render the same underlying classification codes
- They differ only in Layer 1 (User Experience) and, for desktop, in how much of Layer 2 (Health Data) is cached locally



Website Version – Structure

Modules 1–5, 17–21 (UX) over the shared rule-engine backend

- Primary Version 1 channel: fastest to iterate, no install, launches the full end-to-end patient flow
- Registration → profile → blood report upload → rule engine → targeted questions → report → follow-up
- Covers Phase 1 (Metabolic / Cardiovascular) first, per the framework's phased rollout



Website Version – Technology Stack

One language family per side of the stack, minimal moving parts

Layer	Recommendation
Frontend	TypeScript + React (Next.js) – server-rendered marketing pages, client-rendered assessment wizard
Backend / Rule Engine	Python (FastAPI) – strong fit for the rule engine, clean path to ML/OCR, typed schemas via Pydantic
Database	PostgreSQL – Module 25's structured objects map directly to relational tables
Async / background jobs	Python + task queue (e.g. Celery) – OCR extraction and report generation off the request path
Admin console (Module 22)	Same React/TypeScript stack, gated behind medical-admin roles (Module 25)

Website Version – Key Features

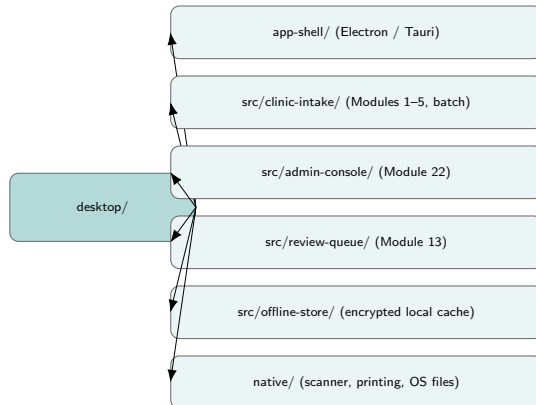
Extraction intelligence, rule fidelity, and admin control

- **LLM/OCR blood-report extraction** – image/PDF upload → OCR → LLM-assisted mapping into structured {test, result, unit, reference range} records against the Lab Test Master Database (Module 6); AI assists extraction and wording only, never classification (framework Section 27)
- Rule-engine API implementing Modules 6–16 exactly as documented, never duplicated client-side
- Unit conversion service (Module 7) so results from different labs are comparable
- Report generator with PDF/print export (Module 19)
- Medical admin console (Module 22) – edit reference ranges, classifications, triggers, recommendation wording without a redeploy
- Auth, consent and role-based access (Modules 1, 25); audit/version logging (Modules 23–24) surfaced per report

Desktop App Version – Structure

Shares the website's UI kit; adds offline intake and admin tooling

- Positioned as the **clinic / medical-professional and administration channel**, not a second consumer app
- Best fit for bulk/administrative rule management, and clinics where connectivity or IT policy favour a local-first tool



Desktop App Version – Technology Stack

Reuses the website's React code; adds a native shell and local store

Layer	Recommendation
Shell	Tauri (Rust shell + the same TypeScript/React UI) – smaller install, lower memory; Electron (Node.js + TS) as fallback for faster native-module access
Shared UI	TypeScript + React – same component library as the website, one design system for both
Local storage	SQLite, encrypted at rest – offline queueing, syncs to PostgreSQL via the same Rule Engine API
Native integrations	Rust (Tauri commands) or Node native modules – scanner / printer access

Desktop App Version – Key Features

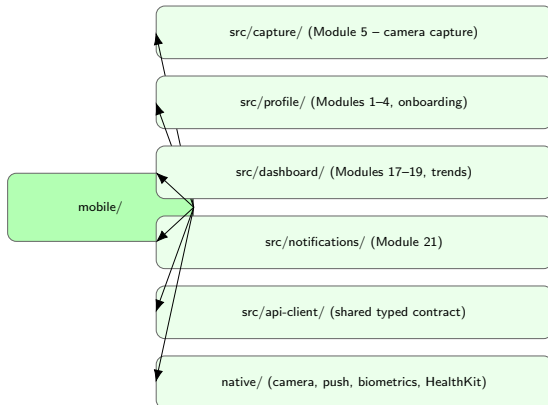
The admin console is arguably desktop's biggest differentiator

- **Offline-first intake** – clinics with unreliable internet register patients and enter/scan reports locally; results sync and pass through the shared Rule Engine once online
- Scanner / printer integration for physical blood reports and for printing the final report (Module 19)
- Batch processing – multiple patients' reports queued and processed together
- Medical admin console (Module 22) as a first-class feature – edit reference ranges, classifications, red flags, recommendation content; review/approve/publish versioning (Module 23) with an audit trail (Module 24)
- Red-flag review queue (Module 13) – dashboard for clinical staff to triage escalated cases
- Role-based permissions distinguishing medical admin, clinic staff and read-only reviewer accounts (Module 25)

Mobile App Version – Structure

Capture-and-track, on the same API contract as web and desktop

- The **consumer companion**: makes the two most-repeated actions frictionless – capturing a new blood report, checking progress
- Carries the longitudinal-tracking and follow-up experience (Modules 17, 20, 21) that a one-time website visit does not serve well



Mobile App Version – Technology Stack

Cross-platform by default, native where capture quality demands it

Layer	Recommendation
App framework	TypeScript + React Native – maximises code/type-sharing with the website's React/TS codebase and API client; Flutter (Dart) is a reasonable alternative if native rendering performance is prioritised over code-sharing
Native modules	Camera / OCR pre-processing (crop, enhance) in Swift/Kotlin invoked from React Native, or a cross-platform camera library where sufficient
Local storage	Encrypted on-device store (e.g. SQLite/Realm) – offline viewing of the last report and cached trends
Push notifications	APNs / FCM – drives Module 21 follow-up and repeat-test reminders

Mobile App Version – Key Features

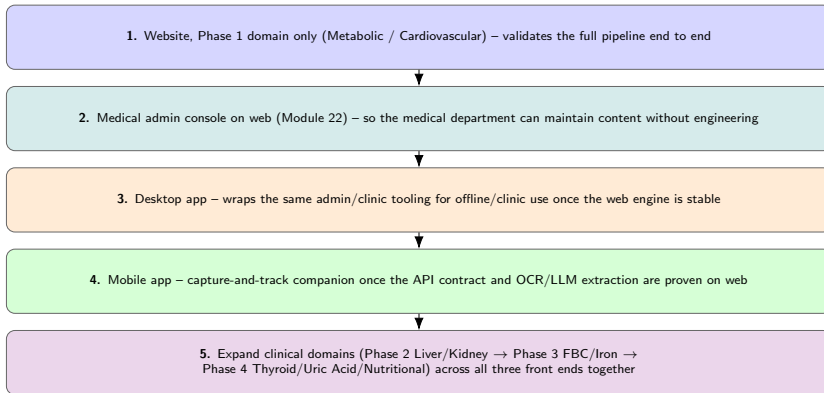
The capture channel that proves OCR/LLM quality daily

- **Camera-based capture + LLM/OCR extraction** – photograph a paper or PDF report, run the same extraction pipeline as the website, confirm low-confidence values (Module 3 validation) before entering the rule engine
- Push notifications for follow-up (Module 21) – “repeat HbA1c in 3 months,” “update your weight,” “your next report is due”
- Trend dashboard (Module 17) – e.g. “Your HbA1c has shown a progressive upward trend across three measurements,” with simple charts
- Biometric login for fast, secure repeat access to sensitive health data
- Optional HealthKit / Health Connect integration – weight, activity, blood pressure feed Modules 3–4 automatically where the user opts in
- Offline report viewing – last report and trend history available without connectivity

Suggested Build Sequence

Web first, then admin tooling, then desktop, then mobile, then scope expansion

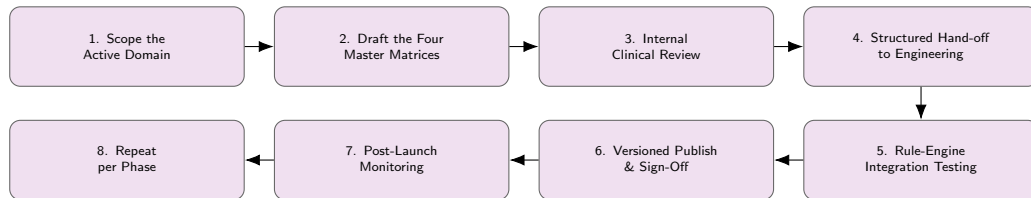
- Follows the framework's own phasing guidance: build one clinical domain at a time, not all 26 modules simultaneously
- Each later clinical-domain expansion only touches the shared rule engine and matrices – not platform-specific work



Medical Department Execution Plan – Eight Steps

Scope → draft → review → hand-off → test → publish → monitor → repeat

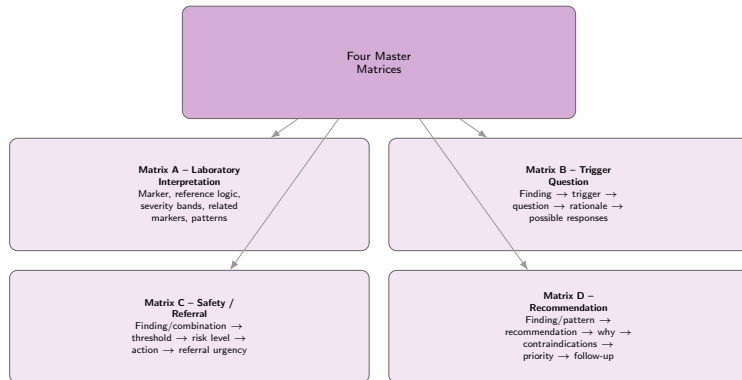
- The medical logic person owns Modules 6–16 plus the medical content of Modules 22–24 – in practice, the Four Master Matrices
- Sequenced so engineering is never blocked on undefined medical content, and nothing ships without clinical sign-off
- The medical department stays one phase ahead of engineering rather than becoming a bottleneck



The Four Master Matrices

The brain of MyHealthReport – Step 2 of the execution plan

- Every matrix entry carries a **RULE_ID**, guideline/source citation and version tag from the outset – this becomes the audit trail in Module 24
- Delivered as structured data (spreadsheet or admin console), never prose – the practical enforcement of “never hard-code medical knowledge into the front end”
- Safety Matrix (C) entries are specifically checked against **Safety Override > Wellness Recommendation**



Thank You

MyHealthReport — Platform Structure & Execution Plan